demo.py

```python

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

# --------------------------------
# 1. Load the Iris dataset
# --------------------------------
iris = load_iris()

X = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)
y = iris.target

df = X.copy()
df["species"] = [iris.target_names[i] for i in y]

print("\n========== DATASET OVERVIEW ==========")
print(df.head())
print("\nDataset shape:", df.shape)
print("\nMissing values:\n", df.isnull().sum())
print("\nSpecies distribution:\n", df["species"].value_counts())

# --------------------------------
# 2. Split the dataset
# --------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# --------------------------------
```

```
# 3. Define classification models
# --------------------------------
models = {
    "Logistic Regression": make_pipeline(
        StandardScaler(),
        LogisticRegression(max_iter=1000)
    ),
    "K-Nearest Neighbors": make_pipeline(
        StandardScaler(),
        KNeighborsClassifier(n_neighbors=5)
    ),
    "Decision Tree": DecisionTreeClassifier(
        max_depth=4,
        random_state=42
    ),
    "Support Vector Machine": make_pipeline(
        StandardScaler(),
        SVC(kernel="linear")
    )
}

# --------------------------------
# 4. Train and evaluate models
# --------------------------------
results = []
trained_models = {}

print("\n========== MODEL EVALUATION ==========")

for name, model in models.items():
    model.fit(X_train, y_train)

    predictions = model.predict(X_test)
    accuracy = accuracy_score(y_test, predictions)

    results.append({
        "Model": name,
        "Accuracy": accuracy
    })

    trained_models[name] = model

    print(f"\n{name}")
    print(f"Accuracy: {accuracy:.2%}")

# --------------------------------
# 5. Compare model performance
# --------------------------------
results_df = pd.DataFrame(results)
results_df = results_df.sort_values(
    by="Accuracy",
    ascending=False
)

```

```
print("\n========== MODEL COMPARISON ==========")
print(results_df.to_string(
    index=False,
    formatters={"Accuracy": "{:.2%}".format}
))

best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]

print("\nBest model on this test split:", best_model_name)

# --------------------------------
# 6. Classification report
# --------------------------------
best_predictions = best_model.predict(X_test)

print("\n========== CLASSIFICATION REPORT ==========")
print(classification_report(
    y_test,
    best_predictions,
    labels=[0, 1, 2],
    target_names=iris.target_names,
    zero_division=0
))

# --------------------------------
# 7. Plot model comparison
# --------------------------------
plt.figure(figsize=(10, 5))
sns.barplot(
    data=results_df,
    x="Accuracy",
    y="Model",
    errorbar=None
)
plt.title("Classification Model Comparison")
plt.xlabel("Accuracy")
plt.xlim(0, 1.05)
plt.tight_layout()
plt.savefig("model_comparison.png", dpi=150)
plt.show()

# --------------------------------
# 8. Confusion matrix
# --------------------------------
cm = confusion_matrix(
    y_test,
    best_predictions,
    labels=[0, 1, 2]
)

plt.figure(figsize=(7, 5))
sns.heatmap(
    cm,
```

```
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.title(f"Confusion Matrix: {best_model_name}")
plt.xlabel("Predicted Species")
plt.ylabel("Actual Species")
plt.tight_layout()
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()

# --------------------------------
# 9. Predict a new flower
# --------------------------------
new_flower = pd.DataFrame(
    [[5.1, 3.5, 1.4, 0.2]],
    columns=iris.feature_names
)

prediction = best_model.predict(new_flower)[0]

print("\n========== NEW PREDICTION ==========")
print("Input features:", new_flower.iloc[0].to_dict())
print("Predicted species:", iris.target_names[prediction])

print("\nProject completed successfully!")


```